

COL100: Introduction to Computer Science

Lab 7: Lists

February 13, 2025

1 Lists

A list is a collection of elements that can be of the same or different data types, stored in a defined order. Lists are dynamic, meaning their size can be adjusted, and they can hold different types of elements. Lists are widely used to manage groups of related data.

1.1 Declaring and Initializing Lists

To declare a list, you simply use square brackets `[]` and specify the elements separated by commas. For example, to declare a list of integers named `numbers`:

```
1 numbers = [2, 4, 6, 8, 10]
```

You can initialize an empty list and add/append elements to it later in the code:

```
1 numbers = []
2 numbers.append(2)
3 numbers.append(4)
```

Note that lists are dynamic and can grow or shrink in size as needed.

1.2 Accessing List Elements

List elements are accessed using indices, starting from 0. To access the third element of the `numbers` list:

```
1 third_element = numbers[2] # Lists are 0-indexed
```

It is important to manage list indices within the valid range to avoid accessing elements outside the list's bounds. Accessing indices that are out of range will raise an `IndexError`.

Always ensure your index values remain within the valid range of the list to maintain program stability and avoid runtime errors.

1.3 Looping Through Lists

Loops are commonly used to iterate through list elements. The `for` loop is often used for this purpose. Here is an example where we loop through elements of a list and print them.

```

1 numbers = [2, 4, 6, 8, 10]
2 for i in range(len(numbers)):
3     print(f"Element at index {i}: {numbers[i]}")

```

1.4 Modifying List Elements

You can modify list elements by assigning new values using their indices:

```

1 numbers[1] = 20 # Changing the second element to 20

```

1.5 Multidimensional Lists

Lists can have more than one dimension. A two-dimensional list is like a table with rows and columns. For example, a 2x3 list:

```

1 matrix = [
2     [1, 2, 3],
3     [4, 5, 6]
4 ]

```

Accessing elements in a 2D list involves specifying both row and column indices. Rows and columns are both 0-indexed:

```

1 element = matrix[1][2] # (value: 6) Accessing element at the second
    row (at index 1) and third column (at index 2)

```

1.6 Finding Maximum and Minimum

Here is an example where we find the minimum and maximum elements in a list.

```

1 numbers = [10, 5, 20, 8, 15]
2
3 # Assume the first element is the maximum and minimum
4 max_value = numbers[0]
5 min_value = numbers[0]
6
7 # Iterate through the list to find the maximum and minimum elements
8 for i in range(1, len(numbers)):
9     if numbers[i] > max_value:
10         max_value = numbers[i] # Update max if current element is
    larger
11     if numbers[i] < min_value:
12         min_value = numbers[i] # Update min if current element is
    smaller
13
14 # Print the maximum and minimum elements
15 print(f"Maximum: {max_value}")
16 print(f"Minimum: {min_value}")

```

2 Submission Problem

2.1 Frequency Count

Given a list of integers, implement a program that counts the frequency of each element and prints the results.

Requirements

- Read an integer n ($1 \leq n \leq 1000$) representing the size of the list.
- Read n integers (non-negative, ≤ 100) representing the list elements.
- Implement a program that calculates and prints the frequency of each element in sorted order.

Sample Input

```
1 8
2 10 20 30 20 40 20 10 30
```

Sample Output

```
1 10 2
2 20 3
3 30 2
4 40 1
```

Sample Input

```
1 10
2 40 23 15 23 40 12 13 15 23 40
```

Sample Output

```
1 12 1
2 13 1
3 15 2
4 23 3
5 40 3
```

3 Practice Problem

3.1 Subarray with Given Sum

Given a list of integers and a target sum, implement a program to check whether there exists a subarray in the given list with the specified sum.

A subarray is a sequence of consecutive elements from the list.

Requirements

- Read an integer **n** ($1 \leq n \leq 100$) representing the size of the list.
- Read **n** integers representing the list elements.
- Read an integer **targetSum** representing the target sum.
- Implement a program that prints *YES* if a subarray exists with the given sum, otherwise print *NO*.

Sample Input

```
1 5
2 1 2 3 4 5
3 9
```

Sample Output

```
1 YES
```

3.2 Rotate an Array

Given a list of integers and a positive integer **k**, implement a program that rotates the elements of the list to the right by **k** positions.

Requirements

- Read an integer **n** ($n \leq 100$) representing the size of the list.
- Read **n** integers representing the list elements.
- Read an integer **k** representing the number of positions to rotate the list to the right.
- Implement a program that rotates the list **in-place** by **k** positions.
- Print the rotated list after the rotation.

Sample Input

```
1 6
2 1 2 3 4 5 6
3 2
```

Sample Output

```
1 5 6 1 2 3 4
```

3.3 Frequency Count

Given a list of integers, you are required to implement a Python program that counts the frequency of each element in the list and prints the results.

- Read an integer n ($n \leq 1000$) denoting the size of the list from the user.
- Read n integers representing the list elements from the user (Each element is a non-negative integer not greater than 100).
- Implement a program that takes the list and its size, and calculates the frequency of each element in the list.
- Print the frequency of each element in a readable format (in sorted order of the elements).

3.3.1 Sample Input and Output

Input

```
1 8
2 1 2 3 2 4 2 1 3
```

Output

```
1 Element 1: 2
2 Element 2: 3
3 Element 3: 2
4 Element 4: 1
```

3.4 Mode of an Array

You are given an array of integers. The mode of an array is the element that appears most frequently in the array. Write a Python program to find the mode of an array of integers.

- Read n ($1 \leq n \leq 1000$), the size of the array.
- Then read n non-negative integers representing the array.
- On a single line, print the mode of the array; if there is no unique most frequent element, then print -1.

Note: You must dynamically handle the creation of lists in your program.

Input

```
1 5
2 1 2 3 4 5
```

Output

```
1 -1
```

Note: There is no unique most frequent element.

Input

```
1 5
2 1 2 2 1 2
```

Output

```
1 2
```

Note: The element 1 appears with frequency 2, and the element 2 appears with frequency 3, hence 2 is the mode of the array.

3.5 Array Reversal

Write a C program to reverse the elements of an integer array in-place i.e. without using an extra array.

- Read, n ($1 \leq n \leq 1000$), the size of the array

- Then read n integers representing the array
- Your program should reverse the elements of the array in-place, i.e. without using an extra array, you can use a fixed number of extra bytes (for eg. one or two extra integers, the number of such extra integers you use should not depend on the size of the array given to you)
- Print the reversed array on a single line as space separated integers

Input

```
1 3
2 1 2 3
```

Output

```
1 3 2 1
```

Input

```
1 6
2 3 2 1 4 5 6
```

Output

```
1 6 5 4 1 2 3
```

3.6 Array Union and Intersection

Write a C program to find the intersection and union of two given integer arrays. Here union and intersection refer to set union and intersection.

- Read $n1$ ($1 \leq n1 \leq 1000$), the size of the first array
- Read $n2$ ($1 \leq n2 \leq 1000$), the size of the second array
- Then read $n1$ integers representing the first array (all the elements will be unique and sorted)
- Then read $n2$ integers representing the second array (all the elements will be unique and sorted)

- First print the intersection of the two arrays, in the sorted order as space separated integers
- Then on the next line print the union of the two arrays, in the sorted order as space separated integers

Input

```
1 4 3
2 1 2 3 4
3 1 2 3
```

Output

```
1 1 2 3
2 1 2 3 4
```

```
1 4 5
2 1 3 4 6
3 3 6 7 8 9
```

Output

```
1 3 6
2 1 3 4 6 7 8 9
```